

1. System Architecture

InvoiceGuard is a full-stack web application that processes invoice submissions through a multi-layer pipeline. The system combines a user-facing frontend, a Node.js backend API, a smart risk scoring logic layer, and an admin monitoring dashboard.

System Flow

Stage	Component	Description
1	User Input	User fills the invoice submission form on the frontend
2	Frontend UI	HTML/CSS form collects and validates all invoice fields
3	Backend API	Express server receives the request and routes it for processing
4	AI + Logic Layer	Risk rules score the invoice and generate an approval decision
5	Database	SQLite stores the invoice details and its decision permanently
6	Admin Dashboard	Monitoring page shows stats, counts, and full invoice history
7	Structured Output	Decision card shown to user with risk badge and reasoning

2. Frontend Implementation

The frontend is a clean, single-page web interface built in Replit. It provides a structured invoice submission form and a dynamic output section that displays the decision result without any raw

Input Form Fields

- Vendor Name: Text input for the supplier or vendor company name
- Invoice Amount: Numeric input for the total invoice value in USD
- Invoice Date: Date picker for when the invoice was issued
- Due Date: Date picker for the payment deadline
- Payment Terms: Dropdown — Net 30, Net 60, Net 90, Immediate
- Vendor History: Dropdown — First-time Vendor, Repeat Vendor, Flagged Vendor

Output Display

After submission, a Decision Result card appears showing: vendor details, invoice amount, dates, payment terms, a colour-coded risk badge (High/Medium/Low), a decision badge (Approved / Review Required / Rejected), and a list of specific risk flags that triggered the assessment.

3. . Logic Layer Implementation

The Smart Logic Layer is the core intelligence of InvoiceGuard. It applies a set of rule-based risk scoring criteria to every invoice, independent of the AI component. This ensures consistent, auditable, and explainable decisions.

Risk Scoring Rules

Rule	Condition	Risk Impact	Decision
Amount Threshold	Invoice > \$500,000	High Risk	Escalate for manual review
Due Date Urgency	Due date < 7 days	Urgent Flag	Review Required
New Vendor	First-time vendor	Medium Risk	Review Required
Extended Terms	Payment terms > Net 60	Medium Risk	Review Required
Flagged Vendor	Vendor history = Flagged	Critical	Auto Reject
All Clear	No rules triggered	Low Risk	Auto Approve

Multiple risk flags can be triggered simultaneously. The highest risk level determines the final decision. All triggered flags are displayed to the user in the Decision Result card.

5. Database Design

InvoiceGuard uses SQLite for persistent storage of all invoice submissions and their decisions. SQLite was chosen for its zero-configuration setup, reliability, and compatibility with the Replit deployment environment.

Invoices Table Schema

Column	Description
vendor_name	Name of the vendor or supplier

amount	Invoice amount in USD
invoice_date	Date the invoice was issued
due_date	Payment due date
payment_terms	Net 30 / Net 60 / Net 90 / Immediate
vendor_history	First-time / Repeat / Flagged
risk_level	Low / Medium / High / Critical
decision	Approved / Review Required / Rejected
risk_flags	JSON array of triggered risk rules
created_at	Timestamp of when invoice was processed

6. Admin Dashboard

The Admin Dashboard provides a real-time monitoring interface for reviewing all processed invoices. It is accessible via the top navigation bar under Admin Dashboard.

Dashboard Features

- Total Invoices: Count of all invoices processed with total dollar value
- Approved: Count of auto-approved low-risk invoices
- Review Required: Count of medium/high risk invoices pending manual review
- Rejected: Count of auto-rejected flagged vendor invoices
- All Invoices Table: Full history with vendor name, amount, risk level, decision, and date
- Colour-coded Badges: Visual indicators for risk level and decision status

7. Git Commit Progression

The project was developed in 6 distinct stages, each tracked with a meaningful Git commit on the GitHub repository. This demonstrates a structured, stage-by-stage development approach.

#	Commit Message	What Was Done
1	Stage 1: Project Setup & Git Initialization	Folder structure, Node.js setup, dependencies installed

2	Stage 2: Frontend UI - Invoice Submission Form	HTML form with all input fields and output display section
3	Stage 3: Backend API - Express Server & Routes	Express server, API endpoint, request handling set up
4	Stage 4: AI Integration & Risk Scoring Logic Layer	Risk rules, scoring logic, and decision mapping implemented
5	Stage 5: Database Integration - SQLite Storage	SQLite schema, save and retrieve invoice records
6	Stage 6: Admin Dashboard - Monitoring & Analytics	Dashboard page with stats cards and invoices table